

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 5

TITLE:

Design and develop a Worker Health and Safety Monitoring System for Construction Industries. The system monitors worker safety parameters and generates alerts during unsafe working conditions.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Construction workers are exposed to various unsafe working conditions such as excessive temperature, high humidity, harmful gases, and abnormal environmental conditions. Continuous monitoring of worker safety parameters is important to reduce risks and improve workplace safety. The proposed system continuously monitors worker health and safety parameters using sensors connected to an ESP32 microcontroller. The collected data can be transmitted through Wi-Fi for remote monitoring, and alerts are generated when unsafe conditions are detected.

PROBLEM STATEMENT

- Exposure of workers to unsafe environmental conditions.
- Lack of continuous worker safety monitoring.
- Delayed detection of dangerous conditions.
- Difficulty in monitoring multiple workers manually.
- Lack of real-time safety information.
- Delayed warning during abnormal conditions.
- Need for automatic safety alerts.

PROPOSED SOLUTION

The proposed system uses an **ESP32 microcontroller and safety-monitoring sensors** to continuously monitor worker-related safety parameters. The ESP32 processes the sensor readings and compares them with predefined safety limits. When an unsafe condition is detected, the system generates an alert using a buzzer and indicator. The monitoring data can also be transmitted through Wi-Fi for remote observation.

WORKING PRINCIPLE

1. Initialize the ESP32.
2. Initialize the worker safety sensors.
3. Connect the ESP32 to Wi-Fi.
4. Read worker safety parameters.
5. Process the sensor readings.
6. Compare the readings with predefined safety limits.
7. Determine the condition as **NORMAL, WARNING, or CRITICAL**.
8. Display the safety readings.
9. Generate an alert when an unsafe condition is detected.
10. Send monitoring data through Wi-Fi.
11. Continue monitoring the worker continuously.

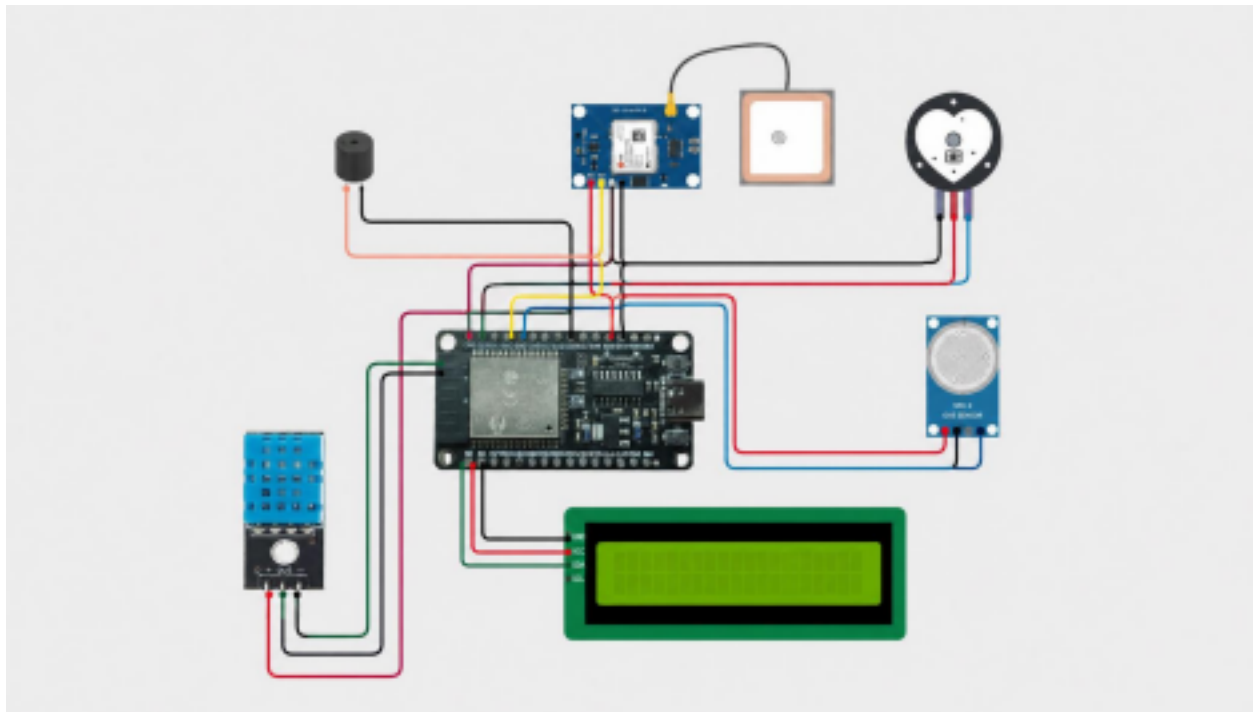
SYSTEM ARCHITECTURE

Safety Sensors → ESP32 Microcontroller → Wi-Fi → Cloud/Monitoring System → Safety Alert

SAFETY PARAMETERS

Parameter	Purpose
Body Temperature	Monitors abnormal worker temperature
Heart Rate	Monitors worker physiological condition
Environmental Temperature	Detects excessive heat
Humidity	Monitors surrounding environmental conditions
Gas/Air Quality	Detects harmful gases or poor air quality
Worker Presence	Identifies worker activity/presence

CIRCUIT DESIGN



ALGORITHM

1. **START**
2. Initialize the ESP32 microcontroller.
3. Initialize the temperature, heart-rate, humidity, and gas sensors.
4. Initialize the LCD, LEDs, and buzzer.
5. Connect the ESP32 to the Wi-Fi network.
6. Read the worker's health and environmental safety parameters.
7. Process the sensor readings.
8. Compare the readings with predefined safety threshold values.
9. Determine the safety condition:
 - a. **NORMAL** – all parameters are within safe limits.
 - b. **WARNING** – a parameter approaches the safety limit.
 - c. **CRITICAL** – a parameter exceeds the safety limit.
10. Display the sensor readings and safety status on the LCD.
11. If the condition is **CRITICAL**, activate the buzzer and warning LED.
12. Send the monitoring data through Wi-Fi for remote monitoring.
13. Repeat the monitoring process continuously.
14. **STOP**

CODING

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include <DHT.h>
#include <TinyGPSPlus.h>
#include <HardwareSerial.h>

//
=====
// WIFI DETAILS
// =====

#define WIFI_SSID "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
```

```

// ADAFRUIT IO DETAILS
// =====

#define IO_USERNAME ""Hair_13"
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// Create Adafruit IO WiFi object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASSWORD
);

//
=====
// FEEDS
// =====

AdafruitIO_Feed *pulseFeed =
  io.feed("pulse");

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");

AdafruitIO_Feed *mq2Feed =
  io.feed("mq2");

AdafruitIO_Feed *latitudeFeed =
  io.feed("latitude");

AdafruitIO_Feed *longitudeFeed =
  io.feed("longitude");

AdafruitIO_Feed *satellitesFeed =
  io.feed("satellites");
AdafruitIO_Feed *gasAlertFeed =
  io.feed("gas-alert");

```

```
//
=====
// DHT11
// =====

#define DHT_PIN 4
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

//
=====
// PULSE SENSOR
//

=====

#define PULSE_PIN 34

//
=====
// MQ2 GAS SENSOR
//

=====

#define MQ2_PIN 35

//
=====
// BUZZER
//

=====

#define BUZZER_PIN 25

//
```

```

=====
// OLED
// =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

//
=====
// GPS
// =====

#define GPS_RX 16
#define GPS_TX 17

HardwareSerial GPS(2);

TinyGPSPlus gps;

//
=====
// VARIABLES
// =====

float temperature = 0;
float humidity = 0;

int pulseValue = 0;

int mq2Value = 0;

float latitude = 0;
float longitude = 0;

int satellites = 0;

```

```

bool gasAlert = false;
//
=====
// TIMING
//

=====

unsigned long lastSend = 0;

const unsigned long sendInterval =
15000;

//
=====
// SETUP
// =====

void setup()
{

    Serial.begin(115200);

    delay(1000);

    // -----
    // DHT
    // -----

    dht.begin();

    // -----
    // SENSOR PINS
    // -----

    pinMode(PULSE_PIN, INPUT);

    pinMode(MQ2_PIN, INPUT);

    pinMode(BUZZER_PIN, OUTPUT);

```



```
digitalWrite(
BUZZER_PIN,
LOW
);

// -----
// OLED
// -----

Wire.begin(21, 22);

if (!display.begin(
SSD1306_SWITCHCAPVCC,
0x3C))
{

Serial.println(
"OLED not detected!"
);
}
else
{

display.clearDisplay();

display.setTextColor(
SSD1306_WHITE
);

display.setTextSize(1);

display.setCursor(10, 10);

display.println(
"SMART MONITOR"
);

display.setCursor(10, 30);
display.println(
"Starting..."
);

display.display();
```

```

}

// -----
// GPS
// -----

GPS.begin(
  9600,
  SERIAL_8N1,
  GPS_RX,
  GPS_TX
);

// -----
// CONNECT TO ADAFRUIT IO //
-----

Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();

// Wait for connection
while (io.status() < AIO_CONNECTED)
{

  Serial.print(".");

  delay(500);
}

Serial.println();

Serial.println(
  "Connected to Adafruit IO!"
);

```

```
display.clearDisplay();
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 10);
```

```
display.println(  
"Adafruit IO"  
);
```

```
display.setCursor(0, 30);
```

```
display.println(  
"Connected!"  
);
```

```
display.display();
```

```
delay(2000);  
}
```

```
//  
=====
```

// LOOP

```
// =====
```

```
void loop()  
{
```

```
// Keep Adafruit IO connection alive  
io.run();
```

```
// -----  
// READ GPS  
// -----
```

```
while (GPS.available())  
{
```

```
gps.encode(  

```

```
GPS.read()  
);  
}
```

```
if (gps.location.isValid())  
{
```

```
latitude =  
gps.location.lat();
```

```
longitude =  
gps.location.lng();  
}
```

```
if (gps.satellites.isValid())  
{
```

```
satellites =  
gps.satellites.value();  
}
```

```
// -----  
// SEND SENSOR DATA  
// -----
```

```
if (
```

```
millis() - lastSend >=  
sendInterval  
)  
{
```

```
lastSend = millis();
```

```
// =====  
// DHT11  
// =====
```

```
temperature =  
dht.readTemperature();
```

```
humidity =  
dht.readHumidity();
```

```
if (isnan(temperature) ||  
isnan(humidity))  
{
```

```
Serial.println(  
"DHT11 reading failed!"  
);
```

```
}  
else  
{
```

```
Serial.print(  
"Temperature: "  
);
```

```
Serial.print(  
temperature  
);
```

```
Serial.println(" C");  
Serial.print(  
"Humidity: "  
);
```

```
Serial.print(  
humidity  
);
```

```
Serial.println(" %");  
}
```

```
// =====  
// PULSE SENSOR  
// =====
```

```
pulseValue =  
analogRead(PULSE_PIN);
```

```
Serial.print(  
  "Pulse Sensor: "  
);
```

```
Serial.println(  
  pulseValue  
);
```

```
// =====  
// MQ2  
// =====
```

```
mq2Value =  
analogRead(MQ2_PIN);
```

```
Serial.print(  
  "MQ2: "  
);
```

```
Serial.println(  
  mq2Value  
);
```

```
// =====  
// GAS ALERT  
// =====
```

```
// Adjust this threshold after testing  
if (mq2Value > 2000)  
{
```

```
  gasAlert = true;
```

```
  digitalWrite(  
    BUZZER_PIN,  
    HIGH  
  );
```

```
}
```

```
else
{

gasAlert = false;

digitalWrite(
BUZZER_PIN,
LOW
);
}


// =====
// GPS SERIAL OUTPUT
// =====
Serial.print(
"Latitude: "
);

Serial.println(
latitude,
6
);

Serial.print(
"Longitude: "
);

Serial.println(
longitude,
6
);

Serial.print(
"Satellites: "
);

Serial.println(
satellites
);
```

```
// =====  
// SEND TO ADAFRUIT IO  
// =====  
  
Serial.println();  
Serial.println(  
"Sending data to Adafruit IO..."  
);  
  
if (!isnan(temperature))  
{  
  
    temperatureFeed->save(  
temperature  
    );  
}  
  
if (!isnan(humidity))  
{  
  
    humidityFeed->save(  
humidity  
    );  
}  
  
pulseFeed->save(  
pulseValue  
    );  
  
mq2Feed->save(  
mq2Value  
    );  
  
latitudeFeed->save(  
latitude  
    );
```



```
longitudeFeed->save(
longitude
);
```

```
satellitesFeed->save(
satellites
);
gasAlertFeed->save(
gasAlert ? 1 : 0
);
```

```
Serial.println(
"Data sent successfully!"
);
```

```
// =====
// OLED
// =====
```

```
display.clearDisplay();
```

```
display.setTextColor(
SSD1306_WHITE
);
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 0);
```

```
display.print("T:");
```

```
display.print(
temperature,
1
);
```

```
display.print("C ");
```

```
display.print("H:");
```

```
display.print(  
humidity,  
0  
);
```

```
display.println("%");
```

```
display.setCursor(0, 15);
```

```
display.print("Pulse: ");
```

```
display.println(  
pulseValue  
);
```

```
display.setCursor(0, 30);
```

```
display.print("MQ2: ");
```

```
display.println(  
mq2Value  
);
```

```
display.setCursor(0, 45);
```

```
if (gasAlert)  
{
```

```
display.println(  
"GAS ALERT!"  
);
```

```
}  
else  
{
```

```
display.println(  

```

```

"Gas Normal"
);
}

display.display();

Serial.println(
"=====
");
}
}

```

SYSTEM WORKING

The Worker Health and Safety Monitoring System is designed using an ESP32 microcontroller and multiple safety sensors. The sensors continuously collect information related to the worker's health and surrounding environmental conditions. The temperature sensor monitors temperature conditions, while the heart-rate sensor monitors the worker's heart rate. The humidity and gas sensors monitor environmental conditions around the worker. The ESP32 receives and processes all sensor readings. The measured values are compared with predefined safety limits. If all parameters are within the safe range, the system indicates NORMAL. When a parameter approaches an unsafe level, the system changes to WARNING. If any parameter exceeds the critical limit, the buzzer and warning LED are activated to alert the worker. The current safety parameters are displayed on the LCD, and the ESP32 can transmit the monitoring data through Wi-Fi for remote observation. The system continuously repeats this process to provide real-time worker safety monitoring.

COMPONENTS

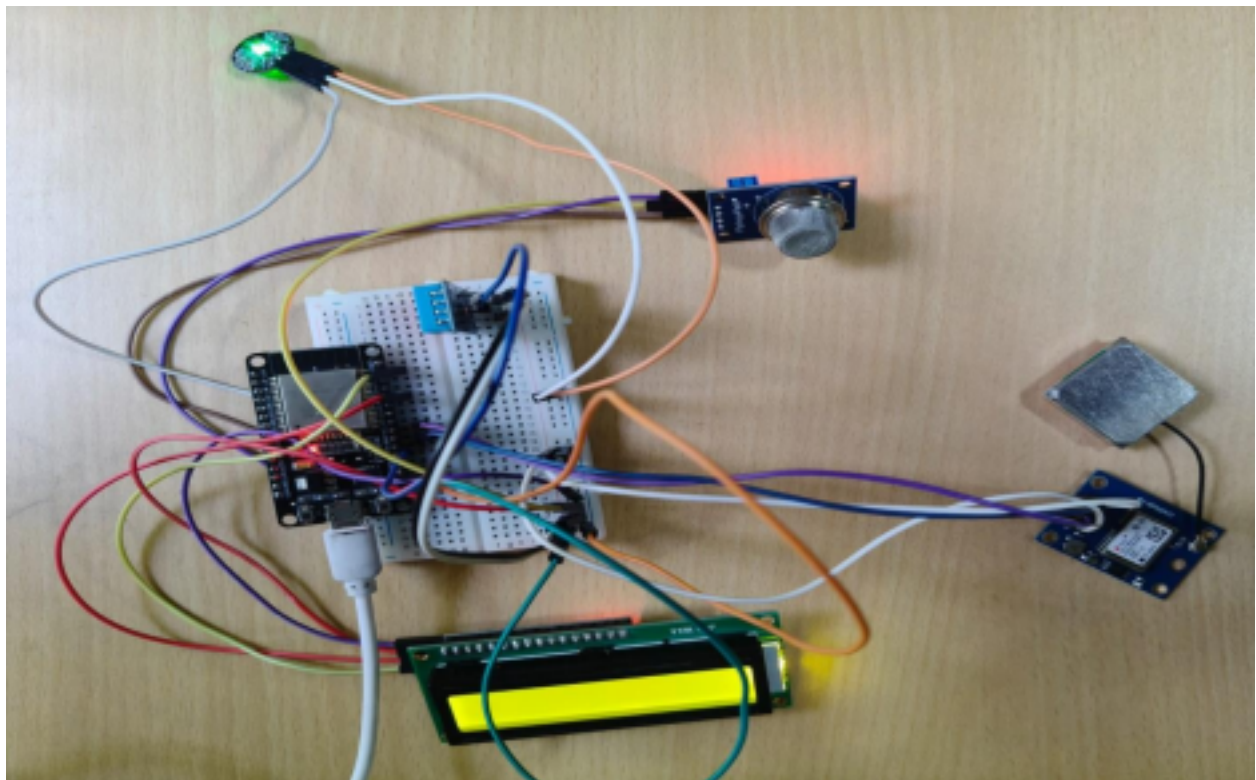
S.No.	Component	Quantity
1	ESP32 Dev Board	1
2	Temperature Sensor	1
3	Heart Rate Sensor	1
4	Gas/Air Quality Sensor	1
5	Humidity Sensor	1
6	16×2 I2C LCD	1

7	Buzzer	1
8	LEDs	2
9	Breadboard & Jumper Wires	1 Set
10	Power Supply	1

PROTOTYPE IMPLEMENTATION

The prototype is designed around an ESP32 microcontroller and multiple worker safety sensors. The temperature, heart-rate, humidity, and gas sensors are connected to the ESP32 to continuously monitor worker health and surrounding environmental conditions. The LCD displays the current safety parameters, while LEDs and a buzzer provide warning indications.

The ESP32 processes the sensor readings and compares them with predefined safety limits. When an unsafe condition is detected, the system changes the safety status to WARNING or CRITICAL and activates the appropriate alert. Wi-Fi connectivity can be used to transmit the monitored safety data for remote observation.



RESULTS & PERFORMANCE ANALYSIS

The developed prototype provides continuous monitoring of worker health and environmental safety parameters. The ESP32 successfully receives sensor readings and evaluates them according to predefined safety limits.

Parameter	Normal Condition	Warning Condition	Critical Condition
Worker Temperature	Within safe range	Approaching limit	Exceeds limit
Heart Rate	Within safe range	Abnormal variation	Critical variation
Humidity	Normal	High/Low	Excessive
Gas/Air Quality	Safe	Elevated	Unsafe

PERFORMANCE

- Provides continuous worker safety monitoring.
- Monitors both health and environmental parameters.
- Detects abnormal conditions automatically.
- Provides local alerts using buzzer and LEDs.
- Displays safety information on the LCD.
- Supports Wi-Fi-based remote monitoring.
- Reduces dependence on manual safety monitoring.
- Helps provide an early warning during unsafe working conditions.
 - The prototype is suitable for educational and small-scale construction safety applications.

